Aran Learn 25: Accessibility, Tenant Isolation and Containers

For: everyone. Inbathamizhan for part 1 (the UI), Aasif first for part 2 (tenant isolation), Arun and Bharadhwaj for part 3 (containers). **Time:** about 1.5 hours with practice. **Before this:** Learn 15 (design tokens), Learn 23 (one UI for the console, Aran Cloud and the banks) and Learn 24 (site structure).

The "improve the whole site" pass checked all 23 pages on a computer and a phone, in dark and light, with an automatic accessibility checker. It also fixed the two problems Learn 23 left open. This doc teaches the three ideas behind those fixes: **accessibility you can measure**, **tenant isolation**, and **containers that start with one command**.

---

1. Accessibility you can measure

Accessibility means people can use the site whatever their eyesight, hands or device. **WCAG 2.1 AA** is the usual standard; banks and governments ask for it. Much of it can be checked by a program. We used **axe-core**, the same engine as the browser's Lighthouse tool:

```
// inside a Playwright page, after the page has loaded
await page.addScriptTag({ path: 'node_modules/axe-core/axe.min.js' });
const r = await page.evaluate(() => axe.run(document, { runOnly: ['wcag2a', 'wcag2aa'] }));
console.log(r.violations.map((v) => v.id));
```

It found three kinds of problem.

Colour contrast

Text must stand out from its background by a **contrast ratio** of at least **4.5 : 1** (3 : 1 for large text). The ratio compares how bright the two colours look: black on white is 21 : 1, and the same colour on itself is 1 : 1.

Where	Before	After	Fix
Red "Erase" and "Dump the database" buttons, dark	white on `#ee5a3f` : 3.4	near-black on `#ee5a3f` : about 6	new token `--on-danger-color`
Teal token pills, light	`#006f66` on its pale tint: 4.09	`#005c55` : above 4.5	darker light-mode token
Red, green and yellow pills, light	4.05 to 4.09	above 4.5	darker light-mode tokens

Because every page takes its colours from `app/ui/css/tokens.css` (Learn 15), changing **four token values** fixed every page in every app at once. That is the payoff of design tokens.

Boxes that scroll sideways

Wide tables and code blocks sit in a box that scrolls sideways on a phone. Someone who uses a keyboard (Tab, then the arrow keys) could not reach that box, so they could never see the hidden columns. The fix is `tabindex="0"` (the box can take focus), `role="region"` and a name, so a screen reader can say "Customers, scrolls sideways". `app/ui/js/ui.js` now does this for every such box on every page, including boxes drawn later, using a **MutationObserver** (a browser feature that tells you when new elements appear).

Things the checker cannot see

Some problems only show up when you look at the page. On a phone:

- **Overlapping rows** in the customer list. The rows sat in a flex column with a fixed height, so the browser squeezed them and long names ran into the next row. Fix: `flex: none` on each row, so rows keep their height and the list scrolls instead.
- **A cut-off language picker** that read "Englis". Fix: on a narrow screen it shows EN, த or हि.
- **The section bar** in Aran Cloud and the banks could hide the current page off to the side. Fix: it scrolls the current page into view.

A clean heading outline

Screen-reader users jump through a page by its headings, like a table of contents. The levels must not skip: the page title is **h1**, its sections are **h2**, and parts of a section are **h3**. Many Aran pages went from h1 straight to h3, because h3 had the size we wanted. The fix keeps the look and corrects the level: `<h2 class="h3">`. **Choose the tag for its meaning and the class for its look.**

Empty states and a path through the site

A page with only zeros tells a first-time visitor nothing. The Overview now shows an **empty state**: one sentence on what will appear, and buttons that make it appear (send six real requests through the firewall, start the normal-traffic replay, or take the tour). Every page also ends with **Previous / Next** links, worked out from the site map (Learn 24), so the site reads as one path from Overview to the design boards.

---

2. Tenant isolation

Aran Cloud serves many banks (**tenants**, Learn 23). Each bank's customer refs were already kept apart with a prefix (`t_ab12__CUST-1`). One leak was left. The core kept a single **owner index**: "this Aadhaar number belongs to customer X". If Bank B sent an Aadhaar number that Bank A had already enrolled, the core answered "already enrolled elsewhere" and filed the token under Bank A's customer. That tells Bank B that the person banks with someone else. It is a privacy leak, and under the DPDP Act it is personal data disclosed without a purpose.

The fix adds a **scope** to the owner index:

```
# research/firewall/shredder.py
def _owner_key(self, typ, value):
    tag = f"own|{typ}|{canonical(typ, value)}" if not self.scope \
          else f"own|{self.scope}|{typ}|{canonical(typ, value)}"
    return hmac.new(self.index_key, tag.encode(), hashlib.sha256).digest()
```

- Aran Cloud sends `"scope": tenant_id` with every `/v1/protect` call.
- The core sets the scope for that one request and clears it afterwards, under the same lock, so one bank's scope can never leak into the next request (`Engine.process`).
- With an empty scope the key is exactly what it was before, so customers already stored in the database are still found. This is called **backward compatibility**.
- Ownership is stored as an **HMAC** (a keyed hash), never as the raw number, so even the index does not hold the Aadhaar number itself.

The test `test_same_identifier_at_two_banks_reveals_nothing` in `aran_cloud/test_cloud_security.py` sends the same Aadhaar number to two workspaces. It checks that Bank B gets no "elsewhere" note, that its token belongs to its own customer, and that it cannot trace Bank A's token.

---

3. Containers that start with one command

A **container** packs a program with everything it needs (Python, libraries, files), so it runs the same on any computer. A **Dockerfile** is the recipe for one image. **Docker Compose** starts several containers together from `docker-compose.yml` .

Before this pass, `docker compose up` failed in four ways:

Problem	**Fix**
The image did not contain Aran Cloud, the demo bank or the SDK	`COPY aran_sdk/ aran_cloud/ demo_bank/` in the Dockerfile
The core's start command pointed at the wrong module, and the cloud's data volume was mounted over its own code	right command and working folder for each service; databases moved to `/srv/data` with `ARAN_CLOUD_DB` and `BANK_DATA_DIR`
The core needs `ARAN_MASTER_KEY` and stopped without it	`scripts/compose_entry.sh` makes random secrets once, on a shared `secrets` volume, unless `.env.local` gives them
Bank B needs an API key that only exists after Aran Cloud is running	`ARAN_AUTO_WORKSPACE=1` : Bank B creates its own workspace on first start and keeps the key on its volume

Now:

```
docker compose up --build
```

starts the core (console on `http://localhost:8000`, published to this computer only), Aran Cloud (`:8100`), Bank A (`:9001`) and Bank B (`:9002`). Each waits for the one before it to pass its **health check** (`depends_on: condition: service_healthy`). The core counts as healthy only once its models have loaded (`"ready":true`).

Volumes keep data when a container is rebuilt. Each service has its own volume, and only the services that need the shared secrets mount the `secrets` volume. That is least privilege again (Learn 22): Bank A, the "legacy" bank, never sees them.

---

4. A live page that tells the truth about its connection

The console asks the server for fresh numbers every few seconds. This is called **polling**. On a free host the server **sleeps** when nobody uses it and **restarts** on every deploy, so the page must handle three cases:

Case	What the page does
A request fails once	Nothing visible. One failure is often just the network
Two rounds fail in a row	LIVE turns to **Reconnecting**, normal polling stops, and only `/api/health` is asked, waiting longer each time (3 s, 4.5 s, ... up to 15 s). Waiting longer after each failure is called **backoff**; it stops a crowd of pages from hammering a server that is trying to start
The server answers with a new `boot` id	It restarted, and its demo data (held in memory) is new. The page says so instead of silently showing zeros

Two more habits: **skip a poll while the last one is still running** (so a slow server never gets a queue of requests), and **stop polling in a hidden tab** (the browser tells you with the `visibilitychange` event).

Exercises

1. Open the light theme and change `--token-color` back to `#006f66`. Run the axe snippet above on Bank B. Which element fails, and with what ratio?
2. Press Tab on the Results page on a phone-sized window until a wide table is focused, then use the arrow keys. What does a screen reader announce?
3. Call the core's `/api/process` twice with the same Aadhaar number, once with `"scope": "bankX"` and once with `"scope": "bankY"`, each with its own `customer_ref`. Do the two answers mention an owner conflict? Now send it twice with no scope and two different refs: what changes?
4. Run `docker compose up --build`, then `docker compose restart bank-b`. Read Bank B's log: why does it not create a second workspace?
5. Open the console, then stop the core server. How long until the bar appears? Start the server again: which bar do you see, and why?

Words to know

Word	Meaning
WCAG 2.1 AA	The common accessibility standard for websites.
Contrast ratio	How much text stands out from its background; 4.5 : 1 or more for normal text.
axe-core	A program that checks a web page against WCAG rules.
Focusable	An element the keyboard can reach with Tab.
Tenant	One customer of a shared service (here, one bank).
Scope	The tenant a lookup is limited to.
Backward compatible	New code still reads data written by old code.
Container	A program packed with everything it needs to run.
Health check	A command that says whether a service is ready.
Volume	Storage that outlives a container.
Polling	Asking the server for fresh data every few seconds.
Backoff	Waiting longer after each failed try.
Boot id	A random id a server picks each time it starts, so clients can tell it restarted.